SQL · BD II

Mapa mental para a prova

Azul: consultas (1 a 9)

Verde: T-SQL (10 a 14)

Ordem: linha 1 → 2 → 3

1 · ORDEM E DICAS

Escrita: SELECT → FROM → JOIN..ON → WHERE → GROUP BY → HAVING → ORDER BY

Execução: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → TOP

Por isso: agregação (SUM, COUNT) vai no **HAVING**, nunca no WHERE; alias do SELECT só vale no ORDER BY.

Roteiro: ① que colunas saem? ② de quais tabelas? ③ como ligam (FK = PK)? ④ filtro de linha → WHERE ⑤ "cada / por" → GROUP BY ⑥ filtro de grupo → HAVING ⑦ ordenar → ORDER BY

Gatilhos do enunciado: "todos" → LEFT JOIN · "não possui" → NOT EXISTS / IS NULL · "no mínimo" → >= · "maior que todos" → ALL · "algum" → ANY

```
SELECT F.Pnome, F.Unome
FROM FUNCIONARIO AS F
JOIN DEPARTAMENTO AS D ON D.Dnumero = F.Dnr
WHERE D.Dnome = 'Pesquisa';
```

Texto entre **aspas simples**; nome dividido: Pnome = 'Joice' AND Unome = 'Leite'.

2 · WHERE · LIKE · BETWEEN · NOT

```
SELECT * FROM FUNCIONARIO AS F
WHERE F.Endereco LIKE '%São Paulo%'
   OR F.Endereco LIKE '%Curitiba%';
```

LIKE: % = qualquer sequência · _ = 1 letra · 'J%' começa com J · '%a' termina com a

```
WHERE F.Dnr = 5
  AND F.Salario BETWEEN 30000 AND 40000;
```

BETWEEN inclui as duas pontas (>= e <=).

Onde vai a negação: antes da condição ou do operador:

```
WHERE NOT F.Endereco LIKE '%São Paulo%'
WHERE F.Endereco NOT LIKE '%São Paulo%'
WHERE F.Salario NOT BETWEEN 30000 AND 40000
WHERE F.Dnr NOT IN (1, 4)
WHERE F.Cpf_supervisor IS NOT NULL
WHERE F.Sexo <> 'M'          -- diferente
```

⚠ "Sem supervisor" = IS NULL (nunca = NULL).

3 · ORDER BY · TOP / LIMIT

ORDER BY é sempre a **última** cláusula. ASC (padrão) = crescente · DESC = decrescente.

```
-- cortar orçamento: maior salário primeiro
SELECT * FROM FUNCIONARIO AS F
ORDER BY F.Salario DESC;

-- vários critérios (desempate)
ORDER BY F.Dnr ASC, F.Salario DESC;
```

TOP n (SQL Server) = **LIMIT n** (MySQL). Sem ORDER BY pega n linhas quaisquer.

```
-- 3 maiores salários
SELECT TOP 3 F.Pnome, F.Salario
FROM FUNCIONARIO AS F
ORDER BY F.Salario DESC;

-- MySQL
SELECT Pnome, Salario FROM FUNCIONARIO
ORDER BY Salario DESC LIMIT 3;
```

DISTINCT remove linhas repetidas: SELECT DISTINCT F.Pnome ...

4 · IN · NOT IN

IN = "é um destes valores" (substitui vários OR).

```
SELECT * FROM FUNCIONARIO AS F
WHERE F.Salario IN (25000, 30000);
```

IN com subconsulta (devolve 1 coluna):

```
SELECT F.Pnome, T.Pnr
FROM FUNCIONARIO F
JOIN TRABALHA_EM T ON F.Cpf = T.Fcpf
WHERE T.Pnr IN (SELECT Pnr FROM TRABALHA_EM
                WHERE Fcpf = '33344555587');
```

NOT IN — funcionários que não são gerentes:

```
SELECT Cpf FROM FUNCIONARIO
WHERE Cpf NOT IN (
   SELECT Cpf_gerente FROM DEPARTAMENTO
   WHERE Cpf_gerente IS NOT NULL);
```

⚠ Se a subconsulta do NOT IN tiver NULL, o resultado volta vazio → filtre IS NOT NULL.

5 · COUNT · SUM · AVG · MAX

```
SELECT COUNT(*) AS Total FROM FUNCIONARIO;
SELECT SUM(Salario) AS CustoMensal
FROM FUNCIONARIO;
SELECT AVG(Salario) AS Media
FROM FUNCIONARIO;
```

COUNT(*) conta linhas · **COUNT(col)** ignora NULL · AVG de INT trunca → AVG(CAST(col AS DECIMAL(10,2))).

Para trazer a **linha** do maior/menor → subconsulta:

```
SELECT * FROM FUNCIONARIO AS F
WHERE F.Salario = (SELECT MAX(Salario)
                   FROM FUNCIONARIO);

-- acima da média
WHERE F.Salario > (SELECT AVG(Salario)
                   FROM FUNCIONARIO);
```

Com GROUP BY, a função é calculada **por grupo** (card 6).

6 · GROUP BY · HAVING

"por / cada X" → **GROUP BY X**. Coluna do SELECT fora de agregação **tem** que estar no GROUP BY.

```
SELECT Sexo, COUNT(*) AS Qtd
FROM FUNCIONARIO
GROUP BY Sexo;

SELECT D.Dnome, SUM(F.Salario) AS Total
FROM DEPARTAMENTO AS D
JOIN FUNCIONARIO AS F ON F.Dnr = D.Dnumero
GROUP BY D.Dnome;
```

HAVING filtra o grupo (depois do GROUP BY); WHERE filtra a linha (antes).

```
SELECT P.Projnome, SUM(T.Horas) AS Horas
FROM PROJETO AS P
JOIN TRABALHA_EM AS T
  ON P.Projnumero = T.Pnr
GROUP BY P.Projnome
HAVING SUM(T.Horas) >= 50;
```

Mais de 3 funcionários: HAVING COUNT(F.Cpf) > 3

7 · UNION · EXCEPT · INTERSECT

Mesmo nº de colunas, tipos compatíveis, ORDER BY só no final.

```
SELECT Dlocal AS Cidade
FROM LOCALIZACAO_DEP
UNION           -- UNION ALL mantém repetidos
SELECT Projlocal FROM PROJETO
ORDER BY Cidade;
```

EXCEPT = está em A e **não** em B:

```
SELECT Cpf FROM FUNCIONARIO
EXCEPT
SELECT Cpf_gerente FROM DEPARTAMENTO
ORDER BY Cpf;
```

INTERSECT = está em A **e** em B:

```
SELECT Cpf FROM FUNCIONARIO
INTERSECT
SELECT Cpf_supervisor FROM FUNCIONARIO;
```

UNION remove repetidos · UNION ALL mantém.

8 · EXISTS · ANY · ALL

EXISTS = existe pelo menos 1 linha; a subconsulta se liga à de fora. **NOT EXISTS** = "não possui".

```
SELECT f.* FROM FUNCIONARIO AS f
WHERE EXISTS (
   SELECT 1 FROM DEPARTAMENTO AS d
   WHERE d.Cpf_gerente = f.Cpf);
```

> ANY = maior que pelo menos um (> mínimo):

```
SELECT f.* FROM FUNCIONARIO AS f
WHERE f.Salario > ANY (
   SELECT f2.Salario FROM FUNCIONARIO f2
   JOIN DEPARTAMENTO d ON f2.Dnr = d.Dnumero
   WHERE d.Dnome = 'Administração');
```

> ALL = maior que todos (> máximo):

```
... GROUP BY p.Projnumero, p.Projnome
HAVING SUM(t.Horas) > ALL (
   SELECT SUM(t2.Horas) FROM PROJETO p2
   JOIN TRABALHA_EM t2 ON t2.Pnr=p2.Projnumero
   WHERE p2.Projlocal = 'Itu'
   GROUP BY p2.Projnumero);
```

9 · INSERT · UPDATE · DELETE

```
INSERT INTO DEPARTAMENTO
   (Dnome, Dnumero, Cpf_gerente)
VALUES ('Marketing', 8, '88866555576');

INSERT INTO FUNCIONARIO
   (Pnome, Unome, Cpf, Salario, Dnr)
VALUES ('Ana', 'Teste', '11111111111',
        3000, NULL);
```

Colunas e valores na **mesma ordem**; coluna omitida fica NULL.

```
UPDATE FUNCIONARIO
SET Salario = Salario * 1.05
WHERE Pnome = 'Joice' AND Unome = 'Leite';

DELETE FROM FUNCIONARIO
WHERE Cpf = '11111111111';
```

⚠ UPDATE/DELETE sem WHERE altera a tabela inteira. Teste antes com SELECT e o mesmo WHERE.

+5% = ×1.05 · +10% = ×1.10 · −20% = ×0.80

10 · DECLARE (variáveis)

Ordem: ① DECLARE ② atribuir (SET ou SELECT) ③ usar ④ GO encerra o lote (a variável some).

```
GO
DECLARE @nome    VARCHAR(50),
        @salario DECIMAL(10,2),
        @novo    DECIMAL(10,2);

SET @nome = 'Jennifer';       -- valor fixo
SELECT @salario = Salario     -- da tabela
FROM FUNCIONARIO WHERE Pnome = @nome;

SET @novo = @salario * 1.10;
PRINT 'Salário de ' + @nome + ' = '
      + CAST(@novo AS VARCHAR(20));
GO
```

Tipos: INT (conta, ano) · DECIMAL(10,2) (dinheiro) · VARCHAR(n) (texto) · CHAR(11) (CPF) · DATE (data) · BIT (sim/não)

Dicas: variável sem valor = NULL · texto + número exige CAST/CONVERT · data BR: CONVERT(VARCHAR(10), @d, 103) · idade: DATEDIFF(YEAR, @nasc, GETDATE())

11 · IF / ELSE · CASE WHEN

IF decide o fluxo do script (fora do SELECT). Dois ou mais comandos → BEGIN ... END.

```
DECLARE @sal DECIMAL(10,2),
        @media DECIMAL(10,2);
SELECT @media = AVG(Salario) FROM FUNCIONARIO;
SELECT @sal = Salario FROM FUNCIONARIO
WHERE Pnome = 'Ana';
IF @sal < @media
   PRINT 'Recebe abaixo da média';
ELSE
   PRINT 'Recebe acima da média';
```

CASE gera um valor **dentro** do SELECT; para no 1º WHEN verdadeiro.

```
SELECT d.Dnome, COUNT(f.Cpf) AS Qtd,
  CASE WHEN COUNT(f.Cpf) <= 1 THEN 'pequeno'
       WHEN COUNT(f.Cpf) <= 3 THEN 'médio'
       ELSE 'grande'
  END AS Tamanho
FROM DEPARTAMENTO AS d
LEFT JOIN FUNCIONARIO AS f ON f.Dnr = d.Dnumero
GROUP BY d.Dnumero, d.Dnome;
```

12 · WHILE

Repete enquanto a condição for verdadeira. **Sempre** altere a variável dentro do laço (senão, loop infinito). BREAK sai do laço.

```
GO
DECLARE @salario DECIMAL(10,2);

SELECT @salario = Salario
FROM FUNCIONARIO
WHERE Pnome = 'Joice' AND Unome = 'Leite';

WHILE @salario < 30000
BEGIN
   SET @salario = @salario * 1.05;
   PRINT 'Novo salário: '
         + CAST(@salario AS VARCHAR(30));
END

UPDATE FUNCIONARIO
SET Salario = @salario
WHERE Pnome = 'Joice' AND Unome = 'Leite';
```

Mudar a variável **não** muda a tabela → precisa do UPDATE no final.

13 · FUNCTION — passo a passo

① GO ② CREATE FUNCTION dbo.nome(@param TIPO) ③ RETURNS tipo *ou* TABLE ④ AS + corpo ⑤ GO ⑥ chamar

Escalar (devolve 1 valor):

```
GO
CREATE FUNCTION dbo.fn_AnoAposentadoria
   (@datanasc DATE)
RETURNS INT
AS
BEGIN
   DECLARE @ano INT;
   SET @ano = YEAR(@datanasc) + 65;
   RETURN @ano;
END
GO
SELECT dbo.fn_AnoAposentadoria('1980-01-01');
```

Tabela (devolve um SELECT):

```
CREATE FUNCTION fn_ProjetosPorFuncionario
   (@cpf VARCHAR(11))
RETURNS TABLE
AS
RETURN (SELECT P.* FROM PROJETO P
   JOIN TRABALHA_EM T ON P.Projnumero = T.Pnr
   WHERE T.Fcpf = @cpf);
GO
SELECT * FROM
 dbo.fn_ProjetosPorFuncionario('12345678966');
```

Função **não** faz INSERT/UPDATE/DELETE nem PRINT.

14 · PROCEDURE — passo a passo

① GO ② CREATE PROCEDURE nome ③ parâmetros @p TIPO (= padrão ou OUTPUT) ④ AS BEGIN ... END ⑤ GO ⑥ EXEC nome valores

```
GO
CREATE PROCEDURE usp_InserirCategoria
   @Tipo VARCHAR(100)
AS
BEGIN
   INSERT INTO Categoria (Tipo_categoria)
   VALUES (@Tipo);
END
GO
EXEC usp_InserirCategoria 'Romance';

CREATE PROCEDURE usp_LivrosPorAno
   @Ano INT
AS
BEGIN
   SELECT * FROM Livro WHERE Ano = @Ano;
END
GO
EXEC usp_LivrosPorAno 2005;
```

Excluir em 2 tabelas: apague a **filha antes do pai**, dentro de BEGIN TRAN ... COMMIT (erro → ROLLBACK).

Função × Procedure: função entra no SELECT e não altera dados; procedure roda com EXEC e pode INSERT/UPDATE/DELETE.